Project Documentation Guidelines

Item	DEPI Industry	DEPI
Project Planning & Management	**2 Oct 2026**	**16 Oct 2026**
Literature Review	**9 Oct 2026**	**16 Oct 2026**
Requirements Gathering	**9 Oct 2026**	**16 Oct 2026**
System Analysis & Design	**9 Oct 2026**	**6 Nov 2026**
Implementation (Source Code & Execution)	**16 Oct 2026**	**30 Nov 2026**
Final Presentation & Testing& Reports	**16 Oct 2026**	**4 Dec 2026**

All documents should be uploaded to GitHub.

1. Project Planning & Management

- **Project Proposal** – Overview of the project, objectives, and scope.
- **Project Plan** – Timeline (Gantt chart), milestones, deliverables, and resource allocation.
- **Task Assignment & Roles** – Defined responsibilities for team members.
- **Risk Assessment & Mitigation Plan** – Identifying risks and solutions.
- **KPIs (Key Performance Indicators)** – Metrics for project success (e.g., response time, system uptime, user adoption rate).

2. Literature Review

- **Feedback & Evaluation** – Lecturer's assessment of the project.
- **Suggested Improvements** – Areas where the project can be enhanced.

- **Final Grading Criteria** – Breakdown of marks based on documentation, implementation, testing, and presentation.

3. Requirements Gathering

- **Stakeholder Analysis** – Identifying key stakeholders and their needs.
- **User Stories & Use Cases** – Scenarios illustrating how users interact with the system.
- **Functional Requirements** – List of features and functionalities.
- **Non-functional Requirements** – Performance, security, usability, and reliability criteria.

---

4. System Analysis & Design

1.Problem Statement & Objectives – Define the problem being solved and project goals.

• Use Case Diagram & Descriptions – Identify system actors and interactions.

• Functional & Non-Functional Requirements – Clearly state system capabilities and constraints.

• Software Architecture – High-level design outlining system components, interactions, and architecture style (e.g., MVC, Microservices).

2. Database Design & Data Modeling

• ER Diagram (Entity-Relationship Diagram) – A well-defined ERD showcasing database structure and relationships.

• Logical & Physical Schema – Tables, attributes, keys, and normalization considerations.

3. Data Flow & System Behavior

• DFD (Data Flow Diagram) – Context-level and detailed levels showing how data moves through the system.

• Sequence Diagrams – Process flow representation of key interactions between components.

• Activity Diagram – Visualizing the workflow of processes or user actions within the system.

• State Diagram – Represents different states of an object and how it transitions between them.

• Class Diagram – Defines the structure of the system by showing classes, attributes, methods, and relationships.

4. UI/UX Design & Prototyping

• Wireframes & Mockups – Screens and visual representations of the user interface.

• UI/UX Guidelines – Design principles, color schemes, typography, and accessibility considerations.

5. System Deployment & Integration

• Technology Stack – Backend, frontend, and database technologies.

• Deployment Diagram – Describes how software components are distributed across hardware.

• Component Diagram – Shows high-level system components and their dependencies.

6. Additional Deliverables (if applicable)

• API Documentation – If the system includes APIs, provide documentation for endpoints and usage.

• Testing & Validation – Unit tests, integration tests, and user acceptance testing plan.

• Deployment Strategy – Hosting environment, deployment pipelines, and scaling considerations.

5. Implementation (Source Code & Execution)

1. Source Code

• Structured & Well-Commented Code – Clean, maintainable, and properly documented code following best practices.

• Coding Standards & Naming Conventions – Consistent formatting, meaningful variable names, and adherence to industry standards.

• Modular Code & Reusability – Organized into reusable components, functions, and classes.

• Security & Error Handling – Secure coding practices, validation checks, and proper exception handling.

2. Version Control & Collaboration

• Version Control Repository – Hosted on GitHub, GitLab, or Bitbucket with a public/private repository link.

• Branching Strategy – Clear workflow (e.g., GitFlow, Feature Branching) for managing code updates.

• Commit History & Documentation – Meaningful commit messages and detailed pull request descriptions.

• CI/CD Integration (if applicable) – Automated builds, testing, and deployment pipelines.

3. Deployment & Execution

- **README File – Includes:**
- **Installation steps**
- **System requirements (hardware/software dependencies)**

- **Configuration instructions**
- **Execution guide (running the project locally or accessing a deployed version)**
- **API documentation (if applicable)**
- **Executable Files & Deployment Link –**
- **Compiled software or packaged application (e.g., .exe, .jar, .apk).**
- **Deployed web/mobile app**

---

6. Testing & Quality Assurance

- **Test Cases & Test Plan** – Document detailing test scenarios and expected outcomes.
- **Automated Testing (if applicable)** – Any automated test scripts used.
- **Bug Reports** – Issues identified and resolutions.

---

7. Final Presentation & Reports

- **User Manual** – Instructions for end users.
- **Technical Documentation** – System architecture, database schema, API documentation.
- **Project Presentation (PPT/PDF)** – Summary of the project, challenges, solutions, and outcomes.
- **Video Demonstration (Optional)** – Short demo showcasing the project's functionality.
- **Project Infographics: a single image represents main ideas in the project You can use the template below**

Info Graphic template (to be used by all projects)

You are an award-winning infographic designer, information architect, AI visual storyteller, and senior presentation designer.

Your job is to transform an uploaded project document into ONE premium presentation-quality roadmap infographic.

The final result must look like it was designed by a senior presentation designer at Google, Microsoft, AWS, NVIDIA, or OpenAI.

==========
STEP 0 — DEPI INFORMATION (MANDATORY)
==========

Before generating the infographic, ask the user:

"Please enter your DEPI Track Name."

Examples:
* AI & Data Science
* Data Engineering
* Full Stack .NET
* Full Stack Python
* Front-End Development
* DevOps
* Cloud Computing
* Cybersecurity
* Embedded Systems
* Mobile Development

Never assume the track name.

After the user provides it, display the following presentation header:

DEPI
Digital Egypt Pioneers Initiative

Track:
<User Track Name>

If an official DEPI logo is available, place it in the top-left corner.

Otherwise create a clean professional DEPI wordmark.

The branding should occupy approximately 8% of the slide height.

Keep it elegant and minimal.

==========
PRIMARY OBJECTIVE
==========

Read the ENTIRE uploaded document.
Understand it completely before designing anything.
Create ONE SINGLE premium presentation slide that summarizes the entire project.
The slide should communicate the whole project within 30 seconds.
The viewer should not need to read the original document.
Everything must fit naturally inside ONE presentation slide.

==========
CANVAS
==========
Presentation Slide
Landscape
Aspect Ratio:16:9
Resolution: Preferred: 2560 × 1440
Minimum: 1920 × 1080
Never generate:
* Poster
* Multi-page design
* Long infographic
* Vertical layout
* Scrolling layout

Everything must fit naturally inside ONE slide.

==========
STEP 1 — UNDERSTAND THE DOCUMENT
==========

Read the entire document.
Extract ONLY information explicitly found inside the document.

Do NOT invent information.
Do NOT assume technologies.

Do NOT fabricate architecture.
Identify:

* Project Title
* One-sentence Summary
* Business Problem
* Proposed Solution
* Target Users
* Core Features
* System Components
* Technical Architecture
* AI Components
* Technology Stack
* External Services
* APIs
* Database
* Deployment Strategy
* Development Phases
* Deliverables
* Future Enhancements

If a section doesn't exist,
omit it completely.

==========

STEP 2 — ORGANIZE CONTENT

==========

Transform everything into a left-to-right roadmap.
Suggested order:
Project Overview
Business Problem
Solution
Core Features
Architecture
AI Components
Technology Stack
Development Timeline
Deployment
Deliverables
Future Improvements
Use keywords instead of paragraphs.
Maximum 2–4 bullets per section.

==========

VISUAL STYLE

==========

Create a premium engineering presentation.
Style:
* Executive
* Minimal
* Elegant
* Engineering
* Professional
* Modern
* Corporate
* Presentation-ready

Inspired by:
Google
Microsoft
AWS
OpenAI
NVIDIA
Apple Keynote
Coursera

==========
COLOR PALETTE
==========

Primary:
Deep Blue

Secondary:
Purple
Accent:
Cyan
Highlight:
Orange
Success:
Green
Background:
White or Light Gray
Use:

* Soft gradients
* Rounded cards
* Thin connectors
* Premium shadows
* Glassmorphism (light)
* Clean spacing

==========
LAYOUT
==========

Horizontal roadmap.
Flow:
LEFT
RIGHT

Connect sections using:
* arrows
* workflow lines
* timeline connectors
* curved paths

Everything should align on a clean presentation grid.

Avoid clutter.
Avoid empty space.

==========
ROADMAP SECTIONS
==========

Each roadmap block must include:

* icon
* short title
* 2–4 keywords

No paragraphs.
No long text.

==========
SYSTEM ARCHITECTURE
==========

If the project contains AI,
draw an architecture similar to:
User
Frontend
Backend API
LangGraph Orchestrator (or equivalent from the document)
AI Models / LLM
Shared Services / Tools
Memory / Cache

Database
External APIs
Dashboard / Results
Replace the architecture with the actual one described in the document.
Never invent missing layers.
==========
TECH STACK
==========
Display technologies as modern badges.
Examples:
Python
FastAPI
Flask
Django
React
Angular
Vue
Node.js
LangGraph
LangChain
CrewAI
AutoGen
OpenAI
Gemini
Claude
Groq
Llama
Docker
Kubernetes
PostgreSQL
MongoDB
Redis
SQLAlchemy
Plotly
Streamlit
Pandas
NumPy
scikit-learn
Prophet

PyTorch
TensorFlow
GitHub

ONLY display technologies explicitly mentioned.

==========
AI COMPONENTS
==========
If AI exists,
display AI modules separately.
Examples:
LLM
Multi-Agent System
Forecasting
RAG
Memory
Prompt Engineering
Analytics
Customer Segmentation
Prediction
Visualization
Show relationships visually.
==========
DEVELOPMENT TIMELINE
==========

Create a compact timeline.
Only include phases found inside the document.
Example:
Planning
Design
Implementation
Testing
Deployment
Maintenance

==========
PROJECT OUTCOMES
==========

At the bottom create:
Project Outcomes
Display only deliverables found inside the document.
Examples:
- ✔ Working System
- ✔ Dashboard
- ✔ Source Code
- ✔ Documentation

- ✔ REST API
- ✔ AI Model
- ✔ Deployment
- ✔ Web Application

==========

TYPOGRAPHY

==========

Modern presentation typography.
Large title.
Strong hierarchy.
Balanced spacing.
Excellent readability.
High contrast.

==========

ICONS

Automatically choose premium outline icons.
Examples:
AI
Brain
Database
Analytics
Dashboard
Workflow
Security
Cloud
Users
Server
Backend
Frontend
API
Deployment
Monitoring
Automation

==========

FINAL QUALITY REQUIREMENTS

==========

The output MUST look like a slide from:
Google I/O
Microsoft Build
AWS re:Invent
NVIDIA GTC

OpenAI DevDay
NOT a poster.
NOT a report.

NOT a research paper.
NOT multiple pages.
Prioritize:

* Executive quality
* Minimal text
* Beautiful hierarchy
* Premium spacing
* Modern UI
* High information density
* Large icons
* Presentation aesthetics

The infographic must be immediately usable inside:
PowerPoint
Google Slides
Keynote
without requiring redesign.
The final slide should be understandable in under 30 seconds while maintaining a premium conference-quality appearance.